

# IMAGE COMPRESSION

## Discrete Fourier Transform

The discrete Fourier transform (DFT) is a type of Fourier transform that allows us to convert discrete data in time domain to a sequence of discrete frequency components effectively giving us a Discrete Time Fourier Transform (DTFT) but discretized in frequency.

DTFT transforms a discrete signal in time to a continuous representation in the frequency domain. However, when processing signals digitally we cannot use continuous representations. The DFT deals with this issue by discretizing the DTFT in the frequency domain. The samples in frequency domain are  $N$  points where  $N$  is also the number of points in the time domain. Similar to how the time samples are spaced by  $T_s$  and are sampled at  $nT_s$  where  $n$  goes from 0 to  $N-1$ , the frequency samples are spaced by  $\frac{f_s}{N}$  and the samples are taken at  $m\frac{f_s}{N}$  where  $m$  goes from 0 to  $N-1$ . The equation for the DFT can be shown below.

$$X(m) = \sum_{n=0}^{N-1} x[n]e^{-2\pi m \frac{n}{N}}$$

In the same way the DFT could be applied in two dimensions such as on images. Where  $X(m_1, m_2)$  would represent the frequency of each axis/dimension. Similar to how the 1-dimensional DFT was derived the 2-dimensional DFT would be defined as follows

$$X(m_1, m_2) = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} x[j, k]e^{-2\pi m_1 \frac{k}{N}}e^{-2\pi m_2 \frac{j}{N}}$$

Where the equation above assumes image  $x[j, k]$  is  $N$  by  $N$ .

## Discrete Cosine Transform

The DFT makes an implicit assumption that the  $N$ -point function we are processing is a periodic  $N$  point function. The periodicity assumption could cause very high frequency components to appear on the borders of each  $N$  point group thus affecting the power of our transformation.

One solution to this problem is to enforce our function to be even symmetric. Making it have  $2N$  samples and making the borders at the groups' edges smoother, reducing the high frequency components. This can also be thought of as enforcing all the coefficients of the sine function be zero such that we are only computing the correlation between our signal and cosine signals instead of complex exponential that include both sine and cosine.

## Experimental Setup

We first setup the project by creating a directory to store the images as well as setting up the parameters. We also create arrays to store the results.

```
% --- Parameters ---
img = im2double(imresize(imread('cameraman.tif'), [256 256]));
blockSizes = [4, 8, 16];
retentionRatios = [0.10, 0.25, 0.50];
outputDir = 'results';
if ~exist(outputDir, 'dir'), mkdir(outputDir); end
% --- Storage for metrics ---
BlockSize = [];
Retention = [];
Compression = [];
MSE_DCT = [];
MSE_DFT = [];
```

In this project we will be testing an image compression method that relies on coefficient reduction.

Before processing the image and compressing it we will split it up into blocks of size 4 by 4, 8 by 8, and 16 by 16. The purpose of splitting it up into blocks is that processing the whole image all at once would be computationally expensive. The below paragraph is a discussion of the compression process which will be done on each block in the image until the whole image is covered. The code below shows a for loop using the function process\_blocks which is what's discussed in the below paragraph.

```
% --- Main processing loop ---
for B = blockSizes
    for ratio = retentionRatios
        % Process blocks
        [imgDCT, totalC, keptC] = process_blocks(img, B, ratio, 'DCT');
        [imgDFT, ~, ~] = process_blocks(img, B, ratio, 'DFT');

        % Clip to [0,1]
        imgDCT = min(max(imgDCT,0),1);
        imgDFT = min(max(imgDFT,0),1);

        % Compute metrics
        mseDCT = mean((img(:)-imgDCT(:)).^2);
        mseDFT = mean((img(:)-imgDFT(:)).^2);
        compressionRatio = totalC / keptC;

        % Store
        BlockSize(end+1) = B;
        Retention(end+1) = ratio;
        Compression(end+1) = compressionRatio;
        MSE_DCT(end+1) = mseDCT;
        MSE_DFT(end+1) = mseDFT;

        % Save images
        fig = figure('Name', sprintf('%dx%d @ %.0f%%', B, B, ratio*100));
        subplot(1,3,1), imshow(img), title('Original');
        subplot(1,3,2), imshow(imgDCT), title(sprintf('DCT (MSE=%.4f)', mseDCT));
        subplot(1,3,3), imshow(imgDFT), title(sprintf('DFT (MSE=%.4f)', mseDFT));
        saveas(fig, sprintf('%s/block_%dx%d_ratio_%02d.png', outputDir, B, B,
round(ratio*100)));
        close(fig);
    end
end
```

```
end  
end
```

We will transform our image using both DFT and DCT. We will use a retention ratio where we will only keep the top 10%, 25%, or 50% of the frequency components while the rest will be set to 0. This will allow us to represent our whole image with just a few frequency components with hopes that the high components will not have much meaningful information in the image. The definitions are shown below in MATLAB.

Then use the Inverse DFT (IDFT) and Inverse DCT (IDCT) to recover the image and compute the mean squared error (MSE) between the reconstructed image and the original image. This method is a lossy compression method since it does not restore the exact original image however it is still incredibly effective and is the basis of the very popular jpeg compression method. The code for the DCT/DFT, retention, and IDCT/IDFT steps performed for each block are shown below.

```
%% === FUNCTION: Process Blocks ===
```

```
function [imgRec, totalCoeffs, keptCoeffs] = process_blocks(img, B, ratio, method)  
    [rows, cols] = size(img);  
    imgRec = zeros(size(img));  
    totalCoeffs = 0;  
    keptCoeffs = 0;  
  
    for i = 1:B:rows  
        for j = 1:B:cols  
            block = img(i:i+B-1, j:j+B-1);  
  
            % Transform  
            switch method  
                case 'DCT'  
                    blockTrans = dct2(block);  
                case 'DFT'  
                    blockTrans = fft2(block);  
                otherwise  
                    error('Unknown method');  
            end  
  
            % Keep top-k coefficients  
            vec = blockTrans(:);  
            k = round(ratio * numel(vec));  
            [~, idx] = sort(abs(vec), 'descend');  
            vecR = zeros(size(vec));  
            vecR(idx(1:k)) = vec(idx(1:k));  
            blockR = reshape(vecR, [B B]);  
  
            % Inverse transform  
            switch method  
                case 'DCT'  
                    imgRec(i:i+B-1, j:j+B-1) = idct2(blockR);  
                case 'DFT'  
                    imgRec(i:i+B-1, j:j+B-1) = real(ifft2(blockR));  
            end  
        end  
    end
```

```

totalCoeffs = totalCoeffs + numel(vec);
keptCoeffs = keptCoeffs + k;
end
end
end

```

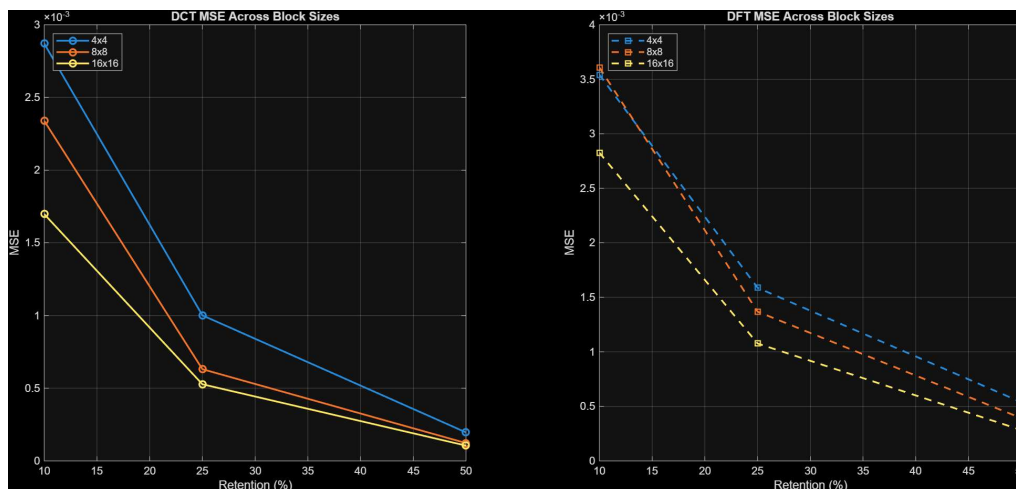
## Results and Interpretation

| Block | Retention | Compression Ratio | MSE_DCT    | MSE_DFT    |
|-------|-----------|-------------------|------------|------------|
| 4     | 0.1       | 8                 | 0.0028702  | 0.0035412  |
| 4     | 0.25      | 4                 | 0.0010011  | 0.0015876  |
| 4     | 0.5       | 2                 | 0.00019605 | 0.00053867 |
| 8     | 0.1       | 10.667            | 0.0023386  | 0.0036075  |
| 8     | 0.25      | 4                 | 0.00063079 | 0.001369   |
| 8     | 0.5       | 2                 | 0.00012262 | 0.00039208 |
| 16    | 0.1       | 9.8462            | 0.0016972  | 0.0028262  |
| 16    | 0.25      | 4                 | 0.00052716 | 0.0010767  |
| 16    | 0.5       | 2                 | 0.0001053  | 0.00028329 |

The first thing to directly note in the results is that the MSE for DCT compression is lower than the MSE for DFT aligning perfectly with the discussion in the first section where we discussed that DCT has less high frequency components and thus the energy is more compact giving better MSE and better compression.

We also note how as the block size increases the MSE decreases for both DCT and DFT since the higher block size allows us to process more of the image at once. Processing the whole 256 by 256 image at once would give a lower MSE but would require higher computational power.

Finally, we note that as the retention ratio increases the MSE error decreases, which makes sense since the higher retention ratio would indicate that we are doing less compression which would in turn result in a higher compression ratio.



The figures above graphically show the discussion above where the graph on the left shows the MSE for DCT against retention ratio for each block size. Similarly, the figure on the right shows the same curve but for DFT. We notice that the range of DCT MSE is less than the range for the DFT. We also note that the curves for each block size have lower values as the block size increases. Finally, we note that for all methods and blocks as the retention ratio increases, the MSE decreases and as the retention ratio becomes too low the MSE gets higher. These results were created using the below code and function.

```
% --- Create results table ---
resultsTable = table(BlockSize(:), Retention(:), Compression(:), MSE_DCT(:),
MSE_DFT(:), ...
    'VariableNames', {'Block','Retention','CompressionRatio','MSE_DCT','MSE_DFT'});
disp(resultsTable);

% --- Plot side-by-side DCT & DFT MSE ---
plot_mse_comparison(BlockSize, Retention, MSE_DCT, MSE_DFT);
%% === FUNCTION: Plot Side-by-Side DCT & DFT MSE ===
function plot_mse_comparison(BlockSize, Retention, MSE_DCT, MSE_DFT)
    BlockSize = BlockSize(:);
    Retention = Retention(:);
    MSE_DCT = MSE_DCT(:);
    MSE_DFT = MSE_DFT(:);
    uniqueBlocks = unique(BlockSize);

    figure;
    % --- Left: DCT ---
    subplot(1,2,1); hold on; grid on; box on;
    for B = uniqueBlocks'
        idx = BlockSize==B;
        plot(Retention(idx)*100, MSE_DCT(idx), '-o','LineWidth',1.6);
    end
    title('DCT MSE Across Block Sizes');
    xlabel('Retention (%)'); ylabel('MSE');

    legend(arrayfun(@(x)sprintf('%dx%d',x,x),uniqueBlocks,'UniformOutput',false),'Location','northwest');

    % --- Right: DFT ---
    subplot(1,2,2); hold on; grid on; box on;
    for B = uniqueBlocks'
        idx = BlockSize==B;
        plot(Retention(idx)*100, MSE_DFT(idx), '--s','LineWidth',1.6);
    end
    title('DFT MSE Across Block Sizes');
    xlabel('Retention (%)'); ylabel('MSE');

    legend(arrayfun(@(x)sprintf('%dx%d',x,x),uniqueBlocks,'UniformOutput',false),'Location','northwest');
end
```

## Running Length and Huffman Encoding (Bonus)

Finally to complete the image compression method, we will add one more detail to the process. After performing coefficient reduction, we will use a running length encoding to decrease the size needed by all the zeros at the end. The image will be read as a vector in a zigzag pattern to make sure the zeros are compacted at the end. Afterwards a Huffman encoding will be used to lossless compress the image even more.

The results for the above compression method are shown below along with the full code.

```
>> RLE_Huffman  
Estimated Compression Ratio = 1.82:1  
Average Bits Per Pixel = 4.3936 bits/pixel
```



```
% == Load & Resize Image ==  
img = im2double(imresize(imread('cameraman.tif'), [256 256]));  
  
% Explicit normalization to [0,1]  
img = (img - min(img(:))) / (max(img(:)) - min(img(:)));  
B = 8;  
ratio = 0.25;  
imgDCT = zeros(size(img));  
  
B = 8;  
ratio = 0.25;  
imgDCT = zeros(size(img));  
  
% Precompute zigzag indices  
zigzagIdx = zigzag_indices(B);  
  
% Storage for encoded data
```

```
encodedBlocks = cell(size(img)/B);

%% === Process Each Block (Compression) ===
for i = 1:B:size(img,1)
    for j = 1:B:size(img,2)
        block = img(i:i+B-1, j:j+B-1);
        dctBlock = dct2(block);

        % --- Keep top-k coefficients ---
        vec = dctBlock(:);
        k = round(ratio * numel(vec));
        [~, idx] = sort(abs(vec), 'descend');
        keepIdx = idx(1:k);
        reduced = zeros(size(vec));
        reduced(keepIdx) = vec(keepIdx);
        dctBlockR = reshape(reduced, [B B]);

        % --- Zigzag Scan ---
        zz = dctBlockR(zigzagIdx);

        % --- Run-Length Encoding ---
        rle = run_length_encode(zz);
        encodedBlocks{(i-1)/B+1, (j-1)/B+1} = rle;
    end
end

%% === Combine All RLE Streams ===
allRLE = [];
for a = 1:numel(encodedBlocks)
    blockRLE = encodedBlocks{a};
    allRLE = [allRLE; blockRLE];
end

% Convert RLE pairs into string symbols for Huffman coding
symbols = cell(size(allRLE,1),1);
for i = 1:length(symbols)
    symbols{i} = sprintf('%g,%g', allRLE(i,1), allRLE(i,2));
end

%% === Build Huffman Dictionary ===
[uniqueSymbols,~,idx] = unique(symbols);
counts = histcounts(idx, 0.5:1:(max(idx)+0.5));
prob = counts / sum(counts);
dict = huffmandict(uniqueSymbols, prob);

%% === Huffman Encoding ===
encodedStream = huffmanenco(symbols, dict);

%% === Compression Statistics ===
original_bits = numel(img) * 8; % assume 8 bits per number
encoded_bits = length(encodedStream);
compression_ratio = original_bits / encoded_bits;

fprintf('Estimated Compression Ratio = %.2f:\n', compression_ratio);
```

```
% === Average Bits Per Pixel ===
num_pixels = numel(img);
bpp = encoded_bits / num_pixels;
fprintf('Average Bits Per Pixel = %.4f bits/pixel\n', bpp);

%% === Huffman Decoding ===
decodedSymbols = huffmandeco(encodedStream, dict);
assert(isequal(symbols, decodedSymbols), 'Huffman decoding mismatch');
disp('Huffman encoding and decoding successful');

%% === Reconstruct All Blocks ===
decodedRLE = zeros(size(allRLE));
for i = 1:length(decodedSymbols)
    vals = sscanf(decodedSymbols{i}, '%f,%f');
    decodedRLE(i,:) = vals';
end

% Split back into cell array per block
decodedBlocks = cell(size(encodedBlocks));
count = 1;
for a = 1:numel(decodedBlocks)
    rle = [];
    while count <= size(decodedRLE,1)
        rle = [rle; decodedRLE(count,:)];
        count = count + 1;
        % heuristic break – each block ends when enough coefficients (B*B)
        if sum(rle(:,2)==0) >= ratio*(B*B)
            break;
        end
    end
    decodedBlocks{a} = rle;
end

%% === RLE Decoding + Inverse DCT ===
imgRec = zeros(size(img));
blockIdx = 1;
for i = 1:B:size(img,1)
    for j = 1:B:size(img,2)
        rle = encodedBlocks{(i-1)/B+1, (j-1)/B+1};
        zz = run_length_decode(rle, B*B);
        blockDCT = zeros(B*B,1);
        blockDCT(zigzagIdx) = zz; % inverse zigzag
        blockDCT = reshape(blockDCT, [B B]);
        block = idct2(blockDCT); % inverse DCT
        imgRec(i:i+B-1, j:j+B-1) = block;
        blockIdx = blockIdx + 1;
    end
end

imgRec = (imgRec - min(imgRec(:))) / (max(imgRec(:)) - min(imgRec(:)));
%% === Display Original vs Reconstructed ===
figure;
subplot(1,2,1);
imshow(img, []);
title('Original Image');
```



```
subplot(1,2,2);
imshow(imgRec, []);
title(sprintf('Reconstructed Image (ratio=%.2f, bpp=%.2f)', ratio, bpp));
%% === Helper Functions ===
function idx = zigzag_indices(n)
    % Generate zigzag scan order indices for an n×n block
    idx = zeros(n,n);
    count = 1;
    for s = 1:2*n-1
        if mod(s,2)==0
            r = max(1, s-n+1):min(n,s);
        else
            r = min(n,s):-1:max(1, s-n+1);
        end
        c = s - r + 1;
        for k = 1:length(r)
            idx(r(k), c(k)) = count;
            count = count + 1;
        end
    end
    [~, order] = sort(idx(:));
    idx = order;
end

function rle = run_length_encode(seq)
    rle = [];
    count = 0;
    for i = 1:length(seq)
        if seq(i) == 0
            count = count + 1;
        else
            if count > 0
                rle = [rle; 0 count];
                count = 0;
            end
            rle = [rle; seq(i) 0];
        end
    end
    if count > 0
        rle = [rle; 0 count];
    end
end

function seq = run_length_decode(rle, N)
    seq = [];
    for i = 1:size(rle,1)
        val = rle(i,1);
        run = rle(i,2);
        if val == 0
            seq = [seq; zeros(run,1)];
        else
            seq = [seq; val];
        end
    end
end
```

```
if length(seq) < N
    seq = [seq; zeros(N - length(seq), 1)];
else
    seq = seq(1:N);
end
end
```